

9913 –Mark Lopes - BE COMPS A | NLP LAB 2

Problems on Weighted Edit Distance (Dynamic Programming)

1. Run the dynamic programming-based algorithm for the (weighted) edit distance problem we saw in class on the following inputs:

- Source string (S = EXCELLENCE)
- Target string (T = SILENCE) ☐

Task:

- Draw the dynamic programming matrix representing the computation. Clearly indicate the
- operations (insert, delete, substitute) carried out to transform the source string into the target string.

2. Run the dynamic programming-based algorithm for the (weighted) edit distance problem we saw in class on the following inputs:

- Source string (S = PARK) Target
- string (T = SPAKE) ☐ **Task:**

- Draw the dynamic programming matrix representing the computation. Clearly indicate the
- operations carried out to convert the source string into the target string.

Refer the link to solve this Practical 2 <https://python.gotrained.com/nltk-editdistance-jaccard-distance/>

When we say that "**character transpositions are not considered**" in **Levenshtein distance**, it means that swapping two adjacent letters (like changing "abcd" to "abdc") is **not treated as a single step**. Instead, it is counted as **two separate operations**, such as two substitutions or a delete and an insert — so the total cost is **2 or more operations**.

For example, to swap 'c' and 'd' in "abcd" , Levenshtein distance would count:

- Substituting 'c' with 'd' and 'd' with 'c' → **2 operations**

On the other hand, **Damerau-Levenshtein distance** does allow **adjacent swaps** as a **single operation**. So the same change from "abcd" to "abdc" would cost only **1 operation** in that case.

Spelling Recommender

For this part of the assignment you will create three different spelling recommenders, that each take a list of misspelled words and recommends a correctly spelled word for every word in the list.

For every misspelled word, the recommender should find the word in `correct_spellings` that has the shortest distance*, and starts with the same letter as the misspelled word, and return that word as a recommendation.

*Each of the three different recommenders will use a different distance measure (outlined below).

Each of the recommenders should provide recommendations for the three default words provided: `['cormulent', 'incendenece', 'validate']`.

Question 1

Task

Your function should generate recommendations for the three given words using the **edit distance metric that includes transpositions** (i.e., **DamerauLevenshtein distance**).

The input consists of three misspelled words:

- `cormulent`
- `incendenece`
- `validate`

Expected Output

The function should return a list containing one corrected recommendation for each word, in the following format:

```
['cormulent_recommendation', 'incendenece_recommendation',  
'validate_recommendation']
```

```
In [ ]: ['cormulent_recommendation', 'incendenece_recommendation', 'validate_recomm  
Out[ ]: ['cormulent_recommendation',  
         'incendenece_recommendation',
```

```

'validate_recommendation'] In

[ ]: # Import the list of English words from the NLTK corpus from
      nltk.corpus import words

      # Import the NLTK library for downloading corpora import
      nltk

      # Import NumPy (optional: used for numerical operations, e.g., distances or
      import numpy as np

      # Download the 'words' corpus from NLTK (only needs to be run once)
      nltk.download('words')

      # Load the list of correct English spellings from the corpus
      correct_spellings = words.words()

```

```

[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data]   Unzipping corpora/words.zip.

```

Student Activity: Fill in the Blanks – Edit Distance with Transpositions

```

In [ ]: # Define a function that recommends correct spellings using edit distance wi
def nltkeditdistance(entries=['cormulent', 'incendenece', 'validate']):

    recommend = []

    for word in entries:

        # Match first letter: filter words in the dictionary that start with
        input_spell = [x for x in correct_spellings if x[0] == word[0]]
        # Hint: Compare the first letter of x and entry

        # Compute Damerau-Levenshtein distance between the word in entries a
        DL_dist = [nltk.edit_distance(word, x, transpositions = True) for x
        #Damerau-Levenshtein distances – a list of edit distances between th
        # Hint: Use transpositions=True

        # Choose the word with the minimum distance
        best_match_index = np.argmin(DL_dist)
        # Hint: This should be the list of distances you've computed

        recommend.append(f"{input_spell[best_match_index]}_recommendation")

    # Hint: Use the filtered list of candidates to retrieve the best mat
    return recommend

# Call the function and print the recommendations print(nltkeditdistance())

```

```

['corpulent_recommendation', 'intendence_recommendation', 'validate_recommen
dation']

```

Question 2

Task

Your function should generate recommendations for the three given words using **Jaccard distance** computed on the **3-grams** (substrings of length 3) of the words.

The input words are: `cormulent`

- `incendenece`
- `validate`
-

Expected Output

The function should return a list containing one corrected recommendation for each word, in the following format:

```
['cormulent_recommendation', 'incendenece_recommendation',  
'validate_recommendation']
```

Student Activity: Fill in the Blanks – Jaccard Distance with Trigrams

In []:

```

# Define a function that recommends correct spellings using Jaccard distance
def jaccard_trigram(entries=['cormulent', 'incendenece', 'validate']):
    recommend = []

    for word in entries:

        # Match first letter: filter words in the dictionary that start with
        input_spell = [x for x in correct_spellings if x[0] == word[0]] #
        Hint: Match first letter of x and word (use indexing like x[0])

        # Compute Jaccard distance between the 3-grams of word and each cand
        jaccard_dist = [nltk.jaccard_distance(set(nltk.ngrams(word, n=3)),
        # Hint: Use trigrams → n = 3

        # Choose the word with the minimum distance
        best_match_index = np.argmin(jaccard_dist)
        # Hint: Fill in the variable that stores Jaccard distances

        recommend.append(f"{input_spell[best_match_index]}_recommendation")
        # Hint: Append the best match from input_spell
    return recommend

# Call the function and print the recommendations print(jaccard_trigram())

['corpulent_recommendation', 'indecence_recommendation', 'validate_recommendation']

```

Question 3

Task

Your function should generate spelling recommendations for the three given words using **Jaccard distance** based on **4-grams** (substrings of length 4).

The input words are: cormulent

- incendenece
- validate
-

Expected Output

The function should return a list containing one recommended correction for each word, in the following format:

```

['cormulent_recommendation', 'incendenece_recommendation',
'validate_recommendation']

```

Student Activity: Fill in the Blanks – Jaccard Distance with 4-grams

In []:

```

# Define a function that recommends correct spellings using Jaccard distance
def jaccard_4_gram(entries=['cormulent', 'incendenece', 'validate']):
    recommend = []

    for word in entries:
        # Match first letter: filter words in the dictionary that start with
        input_spell = [x for x in correct_spellings if x[0] == word[0]]
        # Hint: Use x[0] == word[0] to match the first character

        # Compute Jaccard distance between the 4-grams of word and each cand
        jaccard_dist = [nltk.jaccard_distance(set(nltk.ngrams(word, n=4)),
        # Hint: Use n = 4 for 4-grams

        # Choose the word with the minimum distance
        best_match_index = np.argmin(jaccard_dist)
        # Hint: Use the variable that stores the list of distances

        recommend.append(f"{input_spell[best_match_index]}_recommendation")
        # Hint: Use input_spell to access the word with the smallest distance

    return recommend

# Call the function and print the recommendations print(jaccard_4_gram())

```

['cormus_recommendation', 'incendiary_recommendation', 'valid_recommendation']

Question 4



Task: Custom Edit Distance with Operation Trace

You are required to implement your own function called `editDistance()` that calculates:

- The **minimum edit distance** between two strings.
- A list of **operations** (like insertions, deletions, substitutions) used to transform one string into another.

You will then use this function inside another function named

`LevenshteinDistance_Dynamic()` to generate spelling recommendations for the following misspelled words:

- `cormulent`
- `incendenece`
- `validate`



Expected Output

The `LevenshteinDistance_Dynamic()` function should return a list containing the best recommendations for the input words, based on the minimum edit distance computed using your custom `editDistance()` method:

```
['cormulent_recommendation', 'incendenece_recommendation',
```

```

        #final dp table
    for i in range(1,m+1):
    for j in range(1,n+1):
        if(str1[i-1] == str2[j-1]): #same character
dp[i][j] = dp[i-1][j-1]          ops[i][j] = ops[i-
1][j-1] + ['match']          else:
        delete = dp[i-1][j] + 1          insert = dp[i][j-1] + 1
        substitute = dp[i-1][j-1] + 1          min_cost = min(delete, insert,
substitute) #min cost operation we          dp[i][j] = min_cost

        if(min_cost == delete):
            ops[i][j] = ops[i-1][j-1] + ['delete']
    elif min_cost == insert:
        ops[i][j] = ops[i][j-1] + ['insert']
    else:
        ops[i][j] = ops[i-1][j-1] +
['substitute']    return dp[m][n], dp, ops[m][n]

def LevensteinDistance_Dynamic(entries, correct_spellings):
    recommend = []    paths = []    for entry in entries:
input_spell = [x for x in correct_spellings if x[0] == entry[0]]

        distances = []
    for word in input_spell:
        distance, matrix, operations = levenshtein_distance(entry, word)
        distances.append(distance)
        paths.append((entry, word, matrix, operations))

        min_index = np.argmin(distances)
    recommend.append(input_spell[min_index])    return
recommend, paths

# Example usage
correct_spellings = ["cormorant", "incidence", "validate"] entries
= ["cormulent", "incendenece", "validate"]
recommendations, paths = LevensteinDistance_Dynamic(entries, correct_spellin

print("Recommendations:") print(recommendations)

'validate_recommendation']

```

In []:

)

Recommendations: ['cormorant',
'incidence', 'validate'] 

Instructions

- Answer **all questions** in the notebook only.
 - **Do not upload a PDF file.**
-

Postlab Questions

Q1. Hamming Distance Implementation

Implement the **Hamming distance algorithm** to measure the number of positions at which the corresponding characters are different.

⚠ This is applicable **only for strings of equal length**.

Steps:

1. Write a function `hamming_distance(s1, s2)` that calculates the Hamming distance.
2. Check whether the input strings are of equal length.
3. Count the number of differing characters.
4. Return the Hamming distance, or indicate if the strings are of different lengths.

Sample Input/Output: `hamming_distance("karolin",
"kathrin")` # Output: 3

```
In [ ]: def hamming_distance(s1, s2):      if len(s1) != len(s2):  
return "Error: strings are not of equal length"  
  
        distance = sum(c1 != c2 for c1, c2 in zip(s1, s2)) #count  
return distance  
  
# Sample usage  
print(hamming_distance("karolin", "kathrin"))
```

3

Q.2 Compare Distance Metrics

Compare the results of the different distance metrics for the following string pairs.

Apply all three distance algorithms: **Levenshtein**, **Damerau-Levenshtein**, and **Hamming**.

Also, provide the **strengths and weaknesses** of each metric based on your observations.

○ String Pairs:

1. ("kitten", "sitting")
 2. ("flaw", "lawn")
 3. ("abcdef", "abcfef")
-

G For Each Pair, Report:

- **Levenshtein Distance**
 - **Damerau-Levenshtein Distance**
 - **Hamming Distance** (only if strings are of equal length)
-

💬 Sample Response Format:

Pair: ("kitten", "sitting")

- Levenshtein Distance:
- Damerau-Levenshtein Distance:
- Hamming Distance: Not applicable (different lengths)

Weaknesses:

- **Levenshtein:** Handles insertions/deletions/substitutions well. Cannot handle transpositions.
 - **Damerau-Levenshtein:** Better for cases involving adjacent character swaps.
 - **Hamming:** Simple and fast but limited to equal-length strings.
-

Repeat the format for each pair and include your observations.

ANSWER:

String Pair 1: ("kitten", "sitting")

- **Levenshtein Distance:** 3 Explanation: k→s (substitution), add i, add g
- **Damerau-Levenshtein Distance:** 3 Explanation: Same edits; no adjacent transpositions help here
- **Hamming Distance:** Not applicable (different lengths)

Weaknesses:

- Levenshtein: Handles insertions, deletions, substitutions. Cannot account for transpositions.
- Damerau-Levenshtein: Same here, but would handle adjacent swaps better.
- Hamming: Only works for equal-length strings, so cannot compare here.

String Pair 2: ("flaw", "lawn")

- **Levenshtein Distance:** 2 Explanation: f→l (substitution), add n at the end (or delete/add)
- **Damerau-Levenshtein Distance:** 1 Explanation: Adjacent swap f and l
+ insertion of n counted as single transposition reduces distance

- **Hamming Distance:** Not applicable (different lengths) **Strengths and**

Weaknesses:

- **Levenshtein:** Correctly counts basic edits, but does not treat swaps efficiently.
 - **Damerau-Levenshtein:** Captures adjacent transpositions, more accurate for small swaps.
 - **Hamming:** Cannot use here because lengths differ.
-

String Pair 3: ("abcdef", "abcfef")

- **Levenshtein Distance:** 1 Explanation: d→f (substitution)
- **Damerau-Levenshtein Distance:** 1 Explanation: Same as Levenshtein, no adjacent swap needed
- **Hamming Distance:** 1 Explanation: Only one differing character (d vs f), same length so

Hamming works **Strengths and Weaknesses:**

- **Levenshtein:** Good for insertions, deletions, substitutions
 - **Damerau-Levenshtein:** Handles adjacent swaps too, though not needed here
 - **Hamming:** Fast and simple, works perfectly for equal-length strings, but cannot handle insertions/deletions
-

Distance comparison based on pair

Pair	Levenshtein	Damerau-Levenshtein	Hamming
("kitten", "sitting")	3	3	Not applicable
("flaw", "lawn")	2	1	Not applicable
("abcdef", "abcfef")	1	1	1

Observations

- **Levenshtein:** Good general-purpose metric, simple, handles insertions, deletions, substitutions.
 - **Damerau-Levenshtein:** Extends Levenshtein with transpositions, better for typos with swapped letters.
 - **Hamming:** Fast and intuitive for equal-length strings, but cannot handle insertions/deletions, limiting its applicability.
-